Clustering News Articles

In most of the earlier chapters, we performed data mining knowing what we were looking for. Our use of *target classes* allowed us to learn how our features model those targets during the training phase, which lets the algorithm set internal parameters to maximize its learning. This type of learning, where we have targets to train against, is called **supervised learning**. In this chapter, we'll consider what we do without those targets. This is **unsupervised learning** and it's much more of an exploratory task. Rather than wanting to classify with our model, the goal in unsupervised learning is to explore the data to find insights.

In this chapter, we will look at clustering news articles to find trends and patterns in the data. We'll look at how we can extract data from different websites using a link aggregation website to show a variety of news stories.

The key concepts covered in this chapter include:

- Using the reddit API to collect interesting news stories
- Obtaining text from arbitrary websites
- Cluster analysis for unsupervised data mining
- Extracting topics from documents
- Online learning for updating a model without retraining it
- Cluster ensembling to combine different models

Trending topic discovery

In this chapter, we will build a system that takes a live feed of news articles and groups them together such that the groups have similar topics. You could run the system multiple times over several weeks (or longer) to see how trends change over that time.

Our system will start with the popular link aggregation website (`https://www.reddit.com`), which stores lists of links to other websites, as well as a comments section for discussion. Links on reddit are broken into several categories of links, called **subreddits**. There are subreddits devoted to particular TV shows, funny images, and many other things. What we are interested in are the subreddits for news. We will use the */r/worldnews* subreddit in this chapter, but the code should work with any other text-based subreddit.

In this chapter, our goal is to download popular stories and then cluster them to see any major themes or concepts that occur. This will give us an insight into the popular focus, without having to manually analyze hundreds of individual stories. The general process is to:

1. Collect links to recent popular news stories from reddit.
2. Download the web page from those links.
3. Extract just the news story from the downloaded website.
4. Perform cluster analysis to find clusters of stories.
5. Analyse those clusters to discover trends.

Using a web API to get data

We have used web-based APIs to extract data in several of our previous chapters. For instance, in `Chapter 7`, *Follow Recommendations Using Graph Mining*, we used Twitter's API to extract data. Collecting data is a critical part of the data mining pipeline, and web-based APIs are a fantastic way to collect data on a variety of topics.

There are three things you need to consider when using a web-based API for collecting data: authorization methods, rate limiting, and API endpoints.

Authorization methods allow the data provider to know who is collecting the data, in order to ensure that they are being appropriately rate-limited and that data access can be tracked. For most websites, a personal account is often enough to start collecting data, but some websites will ask you to create a formal developer account to get this access.

Rate limiting is applied to data collection, particularly free services. It is important to be aware of the rules when using APIs, as they can and do change from website to website. Twitter's API limit is 180 requests per 15 minutes (depending on the particular API call). Reddit, as we will see later, allows 30 requests per minute. Other websites impose daily limits, while others limit on a per-second basis. Even within websites, there are drastic differences for different API calls. For example, Google Maps has smaller limits and different API limits per-resource, with different allowances for the number of requests per hour.

If you find you are creating an app or running an experiment that needs more requests and faster responses, most API providers have commercial plans that allow for more calls. Contact the provider for more details.

API Endpoints are the actual URLs that you use to extract information. These vary from website to website. Most often, web-based APIs will follow a RESTful interface (short for **Representational State Transfer**). RESTful interfaces often use the same actions that HTTP does: `GET`, `POST`, and `DELETE` are the most common. For instance, to retrieve information on a resource, we might use the following (example only) API endpoint: `www.dataprovider.com/api/resource_type/resource_id/`

To get the information, we just send an HTTP `GET` request to this URL. This will return information on the resource with the given type and ID. Most APIs follow this structure, although there are some differences in the implementation. Most websites with APIs will have them appropriately documented, giving you details of all the APIs that you can retrieve.

First, we set up the parameters to connect to the service. To do this, you will need a developer key for reddit. In order to get this key, log into the site at `https://www.reddit.com/login` and go to `https://www.reddit.com/prefs/apps`. From here, click on are you a developer? create an app... and fill out the form, setting the type as script. You will get your client ID and a secret, which you can add to a new Jupyter Notebook:

```
CLIENT_ID = "<Enter your Client ID here>"
CLIENT_SECRET = "<Enter your Client Secret here>"
```

Reddit also asks you (when you use their API) to set the user agent to a unique string that includes your username. Create a user agent string that uniquely identifies your application. I used the name of the book, chapter 10, and a version number of 0.1 to create my user agent, but it can be any string you like. Note that not doing this may result in your connection being heavily rate-limited:

```
USER_AGENT = "python:<your unique user agent> (by /u/<your reddit username>)"
```

In addition, you will need to log in to reddit using your username and password. If you don't have one already, sign up for a new one (it is free and you don't need to verify with personal information either).

You will need your password to complete the next step, so be careful before sharing your code to others to remove it. If you don't put your password in, set it to none and you will be prompted to enter it.

Now let's create the username and password:

```
from getpass import getpass
USERNAME = "<your reddit username>"
PASSWORD = getpass("Enter your reddit password:")
```

Next, we are going to create a function to log with this information. The reddit login API will return a token that you can use for further connections, which will be the result of this function. The code obtains the necessary information to log in to reddit, set the user agent, and then obtain an access token that we can use with future requests:

```
import requests
def login(username, password):
    if password is None:
        password = getpass.getpass("Enter reddit password for user {}: ".format(username))
    headers = {"User-Agent": USER_AGENT}
    # Setup an auth object with our credentials
    client_auth = requests.auth.HTTPBasicAuth(CLIENT_ID, CLIENT_SECRET)
    # Make a post request to the access_token endpoint
    post_data = {"grant_type": "password", "username": username, "password": password}
    response = requests.post("https://www.reddit.com/api/v1/access_token", auth=client_auth,

                             data=post_data, headers=headers)
    return response.json()
```

We can call now our function to get an access token:

```
token = login(USERNAME, PASSWORD)
```

This token object is just a dictionary, but it contains the `access_token` string that we will pass along with future requests. It also contains other information such as the scope of the token (which would be everything) and the time in which it expires, for example:

```
{'access_token': '<semi-random string>', 'expires_in': 3600, 'scope': '*', 'token_type': 'beare
```

If you are creating a production-level app, make sure you check the expiry of the token and to refresh

it if it runs out. You'll also know this has happened if your access token stops working when trying to make an API call.

Reddit as a data source

Reddit is a link aggregation website used by millions worldwide, although the English versions are US-centric. Any user can contribute a link to a website they found interesting, along with a title for that link. Other users can then upvote it, indicating that they liked the link, or downvote it, indicating they didn't like the link. The highest voted links are moved to the top of the page, while the lower ones are not shown. Older links are removed from the front page over time, depending on how many upvotes it has. Users who have stories upvoted earn points called karma, providing an incentive to submit only good stories.

Reddit also allows non-link content, called self-posts. These contain a title and some text that the submitter enters. These are used for asking questions and starting discussions. For this chapter, we will be considering only link-based posts, and not comment-based posts.

Posts are separated into different sections of the website called subreddits. A subreddit is a collection of posts that are related. When a user submits a link to reddit, they choose which subreddit it goes into. Subreddits have their own administrators, and have their own rules about what is valid content for that subreddit.

By default, posts are sorted by **Hot**, which is a function of the age of a post, the number of upvotes, the number of downvotes it has received and how liberal the content is. There is also **New**, which just gives you the most recently posted stories (and therefore contains lots of spam and bad posts), and **Top**, which is the highest voted stories for a given time period. In this chapter, we will be using Hot, which will give us recent, higher-quality stories (there really are a lot of poor-quality links in New).

Using the token we previously created, we can now obtain sets of links from a subreddit. To do that, we will use the /r/<subredditname> API endpoint that, by default, returns the Hot stories. We will use the /r/worldnews subreddit:

```
subreddit = "worldnews"
```

The URL for the previous endpoint lets us create the full URL, which we can set using string formatting:

```
url = "https://oauth.reddit.com/r/{}".format(subreddit)
```

Next, we need to set the headers. This is needed for two reasons: to allow us to use the authorization token we received earlier and to set the user agent to stop our requests from being heavily restricted. The code is as follows:

```
headers = {"Authorization": "bearer {}".format(token['access_token']),
"User-Agent": USER_AGENT}
```

Then, as before, we use the requests library to make the call, ensuring that we set the headers:

```
response = requests.get(url, headers=headers)
```

Calling `json()` on this will result in a Python dictionary containing the information returned by reddit. It will contain the top 25 results from the given subreddit. We can get the title by iterating over the stories in this response. The stories themselves are stored under the dictionary's data key. The code is as follows:

```
result = response.json()
for story in result['data']['children']:
    print(story['data']['title'])
```

Getting the data

Our dataset is going to consist of posts from the Hot list of the /r/worldnews subreddit. We saw in the previous section how to connect to reddit and how to download links. To put it all together, we will create a function that will extract the titles, links, and score for each item in a given subreddit.

We will iterate through the subreddit, getting a maximum of 100 stories at a time. We can also do pagination to get more results. We can read a large number of pages before reddit will stop us, but we will limit it to 5 pages.

As our code will be making repeated calls to an API, it is important to remember to rate-limit our calls. To do so, we will need the sleep function:

```
from time import sleep
```

Our function will accept a subreddit name and an authorization token. We will also accept a number of pages to read, though we will set a default of 5:

```
def get_links(subreddit, token, n_pages=5):
    stories = []
    after = None
    for page_number in range(n_pages):
        # Sleep before making calls to avoid going over the API limit
        sleep(2)
        # Setup headers and make call, just like in the login function
        headers = {"Authorization": "bearer {}".format(token['access_token']), "User-Agent": USI

        url = "https://oauth.reddit.com/r/{}?limit=100". format(subreddit)
        if after:
            # Append cursor for next page, if we have one
            url += "&after={}".format(after)
        response = requests.get(url, headers=headers)
        result = response.json()
        # Get the new cursor for the next loop
        after = result['data']['after']
        # Add all of the news items to our stories list
        for story in result['data']['children']:
            stories.append((story['data']['title'], story['data']['url'], story['data']['score']

    return stories
```

We saw in Chapter 7, Follow Recommendations Using Graph Mining, how pagination works for the Twitter API. We get a cursor with our returned results, which we send with our request. Twitter will then use this cursor to get the next page of results. The reddit API does almost exactly the same thing, except it calls the parameter after. We don't need it for the first page, so we initially set it to None. We will set it to a meaningful value after our first page of results.

Calling the stories function is a simple case of passing the authorization token and the subreddit name:

```
stories = get_links("worldnews", token)
```

The returned results should contain the title, URL, and 500 stories, which we will now use to extract the actual text from the resulting websites. Here is a sample of the titles that I received by running the script:

Russia considers banning sale of cigarettes to anyone born after 2015
Swiss Muslim girls must swim with boys
Report: Russia spread fake news and disinformation in Sweden - Russia has coordinated a campaign over the past 2years to influence Sweden's decision making by using disinformation, propaganda and false documents, according to a report by researchers at The Swedish Institute of International Affairs.
100% of Dutch Trains Now Run on Wind Energy. The Netherlands met its renewable energy goals a year ahead of time.
Legal challenge against UK's sweeping surveillance laws quickly crowdfunded
A 1,000-foot-thick ice block about the size of Delaware is snapping off of Antarctica
The U.S. dropped an average of 72 bombs every day — the equivalent of three an hour — in 2016, according to an analysis of American strikes around the world. U.S. Bombed Iraq, Syria, Pakistan, Afghanistan, Libya, Yemen, Somalia in 2016
The German government is investigating a recent surge in fake news following claims that Russia is attempting to meddle in the country's parliamentary elections later this year.
Pesticides kill over 10 million bees in a matter of days in Brazil countryside
The families of American victims of Islamic State terrorist attacks in Europe have sued Twitter, charging that the social media giant allowed the terror group to proliferate online
Gas taxes drop globally despite climate change; oil & gas industry gets $500 billion in subsidies; last new US gas tax was in 1993
Czech government tells citizens to arm themselves and shoot Muslim terrorists in case of 'Super Holocaust'
PLO threatens to revoke recognition of Israel if US embassy moves to Jerusalem
Two-thirds of all new HIV cases in Europe are being recorded in just one country – Russia: More than a million Russians now live with the virus and that number is expected to nearly double in the next decade
Czech government tells its citizens how to fight terrorists: Shoot them yourselves | The interior ministry is pushing a constitutional change that would let citizens use guns against terrorists
Morocco Prohibits Sale of Burqa
Mass killer Breivik makes Nazi salute at rights appeal case
Soros Groups Risk Purge After Trump's Win Emboldens Hungary
Nigeria purges 50,000 'ghost workers' from State payroll in corruption sweep
Alcohol advertising is aggressive and linked to youth drinking, research finds | Society
UK Government quietly launched 'assault on freedom' while distracting people, say campaigners behind legal challenge - The Investigatory Powers Act became law at the end of last year, and gives spies the power to read through everyone's entire internet history
Russia's Reserve Fund down 70 percent in 2016
Russian diplomat found dead in Athens
At least 21 people have been killed (most were civilians) and 45 wounded in twin bombings near the Afghan parliament in Kabul
Pound's Decline Deepens as Currency Reclaims Dubious Honor

World news isn't usually the most optimistic of places, but it does give insight into what is going on around the world, and trends on this subreddit are usually indicative of trends in the world.

Extracting text from arbitrary websites

The links that we get from reddit go to arbitrary websites run by many different organizations. To make it harder, those pages were designed to be read by a human, not a computer program. This can cause a problem when trying to get the actual content/story of those results, as modern websites have a lot going on in the background. JavaScript libraries are called, style sheets are applied, advertisements are loaded using AJAX, extra content is added to sidebars, and various other things are done to make the modern web page a complex document. These features make the modern Web what it is, but make it difficult to automatically get good information from!

Finding the stories in arbitrary websites

To start with, we will download the full web page from each of these links and store them in our data folder, under a raw subfolder. We will process these to extract the useful information later on. This caching of results ensures that we don't have to continuously download the websites while we are working. First, we set up the data folder path:

```
import os
data_folder = os.path.join(os.path.expanduser("~"), "Data", "websites", "raw")
```

We are going to use MD5 hashing to create unique filenames for our articles, by hashing the URL, and we will import `hashlib` to do that. A `hash` function is a function that converts some input (in our case a string containing the title) into a string that is seemingly random. The same input will always return the same output, but slightly different inputs will return drastically different outputs. It is also impossible to go from a hash value to the original value, making it a one-way function.

```
import hashlib
```

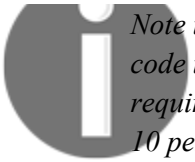
For this chapter's experiments, we are going to simply skip any website downloads that fail. In order to make sure we don't lose too much information doing this, we maintain a simple counter of the number of errors that occur. We are going to suppress any error that occurs, which could result in a systematic problem prohibiting downloads. If this error counter is too high, we can look at what those errors were and try to fix them. For example, if the computer has no internet access, all 500 of the downloads will fail and you should probably fix that before continuing!

```
number_errors = 0
```

Next, we iterate through each of our stories, download the website, and save the results to a file:

```
for title, url, score in stories:
    output_filename = hashlib.md5(url.encode()).hexdigest()
    fullpath = os.path.join(data_folder, output_filename + ".txt")
    try:
        response = requests.get(url)
        data = response.text
        with open(fullpath, 'w') as outf:
            outf.write(data)
        print("Successfully completed {}".format(title))
    except Exception as e:
        number_errors += 1
        # You can use this to view the errors, if you are getting too many:
        # raise
```

If there is an error in obtaining the website, we simply skip this website and keep going. This code will work on a large number of websites and that is good enough for our application, as we are looking for general trends and not exactness.



Note that sometimes you do care about getting 100 percent of responses, and you should adjust your code to accommodate more errors. Be warned though that there is a significant increase in effort required to create code the works reliably on data from the internet. The code to get those final 5 to 10 percent of websites will be significantly more complex.

In the preceding code, we simply catch any error that happens, record the error and move on.

If you find that too many errors occur, change the print(e) line to just type raise instead. This will cause the exception to be called, allowing you to debug the problem.

After this has completed, we will have a bunch of websites in our `raw` subfolder. After taking a look at these pages (open the created files in a text editor), you can see that the content is there but there is HTML, JavaScript, CSS code, as well as other content. As we are only interested in the story itself, we now need a way to extract this information from these different websites.

Extracting the content

After we get the raw data, we need to find the story in each. There are several complex algorithms for doing this, as well as some simple ones. We will stick with a simple method here, keeping in mind that often enough, the simple algorithm is good enough. This is part of data mining—knowing when to use simple algorithms to get a job done, versus using more complicated algorithms to obtain that extra bit of performance.

First, we get a list of each of the filenames in our `raw` subfolder:

```
filenames = [os.path.join(data_folder, filename) for filename in os.listdir(data_folder)]
```

Next, we create an output folder for the text-only versions that we will extract:

```
text_output_folder = os.path.join(os.path.expanduser("~"), "Data", "websites", "textonly")
```

Next, we develop the code that will extract the text from the files. We will use the lxml library to parse the HTML files, as it has a good HTML parser that deals with some badly formed expressions. The code is as follows:

```
from lxml import etree
```

The actual code for extracting text is based on three steps:

1. We iterate through each of the nodes in the HTML file and extract the text in it.
2. We skip any node that is JavaScript, styling, or a comment, as this is unlikely to contain information of interest to us.
3. We ensure that the content has at least 100 characters. This is a good baseline, but it could be improved upon for more accurate results.

As we said before, we aren't interested in scripts, styles, or comments. So, we create a list to ignore nodes of those types. Any node that has a type in this list will not be considered as containing the story. The code is as follows:

```
skip_node_types = ["script", "head", "style", etree.Comment]
```

We will now create a function that parses an HTML file into an lxml `etree`, and then we will create another function that parses this tree looking for text. This first function is pretty straightforward; simply open the file and create a tree using the lxml library's parsing function for HTML files. The code is as follows:

```
parser = etree.HTMLParser()
```

can't get good results, or need better results for your application, try the methods listed in *Appendix A, Next Steps.*

```
def get_text_from_file(filename):
    with open(filename) as inf:
        html_tree = etree.parse(inf, parser)
    return get_text_from_node(html_tree.getroot())
```

In the last line of that function, we call the `getroot()` function to get the root node of the tree, rather than the full `etree`. This allows us to write our text extraction function to accept any node, and therefore write a recursive function.

This function will call itself on any child nodes to extract the text from them, and then return the concatenation of any child nodes text.

If the node where this function is passed doesn't have any child nodes, we just return the text from it. If it doesn't have any text, we just return an empty string. Note that we also check here for our third condition—that the text is at least 100 characters long.

The code for checking that the text is at least 100 characters long is as follows:

```
def get_text_from_node(node):
    if len(node) == 0:
        # No children, just return text from this item
        if node.text:
            return node.text
        else:
            return ""
    else:
        # This node has children, return the text from it:
        results = (get_text_from_node(child)
                   for child in node
                   if child.tag not in skip_node_types)
    result = str.join("n", (r for r in results if len(r) > 1))
    if len(result) >= 100:
        return result
    else:
        return ""
```

At this point, we know that the node has child nodes, so we recursively call this function on each of those child nodes and then join the results when they return.

The final condition inside the return line stops blank lines being returned (for example, when a node has no children and no text). We also use a generator, which makes the code more efficient by only grabbing text data when it is needed, namely the final return statement rather than creating a number of sub-lists.

We can now run this code on all of the raw HTML pages by iterating through them, calling the text extraction function on each, and saving the results to the text-only subfolder:

```
for filename in os.listdir(data_folder):
    text = get_text_from_file(os.path.join(data_folder, filename))
    with open(os.path.join(text_output_folder, filename), 'w') as outf:
        outf.write(text)
```

You can evaluate the results manually by opening each of the files in the text only subfolder and checking their content. If you find too many of the results have non-story content, try increasing the minimum-100-character limit. If you still

Grouping news articles

The aim of this chapter is to discover trends in news articles by clustering, or grouping, them together. To do that, we will use the k-means algorithm, a classic machine learning algorithm originally developed in 1957.

Clustering is an unsupervised learning technique and we often use clustering algorithms for exploring data. Our dataset contains approximately 500 stories and it would be quite arduous to examine each of those stories individually. Using clustering allows us to group similar stories together, and we can explore the themes in each cluster independently.

We use clustering techniques when we don't have a clear set of target classes for our data. In that sense, clustering algorithms have little direction in their learning. They learn according to some function, regardless of the underlying meaning of the data.

For this reason, it is critical to choose good features. In supervised learning, if you choose poor features, the learning algorithm can choose to not use those features. For instance, support vector machines will give little weight to features that aren't useful in classification. However, with clustering, all features are used in the final result—even if those features don't provide us with the answer we were looking for.

When performing cluster analysis on real-world data, it is always a good idea to have a sense of what sorts of features will work for your scenario. In this chapter, we will use the bag-of-words model. We are looking for topic-based groups, so we will use topic-based features to model the documents. We know those features work because of the work others have done in supervised versions of our problem. In contrast, if we were to perform an authorship-based clustering, we would use features such as those found in the `Chapter 9`*, Authorship Attribution*, experiment.

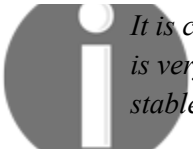
The k-means algorithm

The k-means clustering algorithm finds centroids that best represent the data using an iterative process. The algorithm starts with a predefined set of centroids, which are normally data points taken from the training data. The **k** in k-means is the number of centroids to look for and how many clusters the algorithm will find. For instance, setting k to 3 will find three clusters in the dataset.

There are two phases to the k-means: **assignment** and **updating**. They are explained as below:

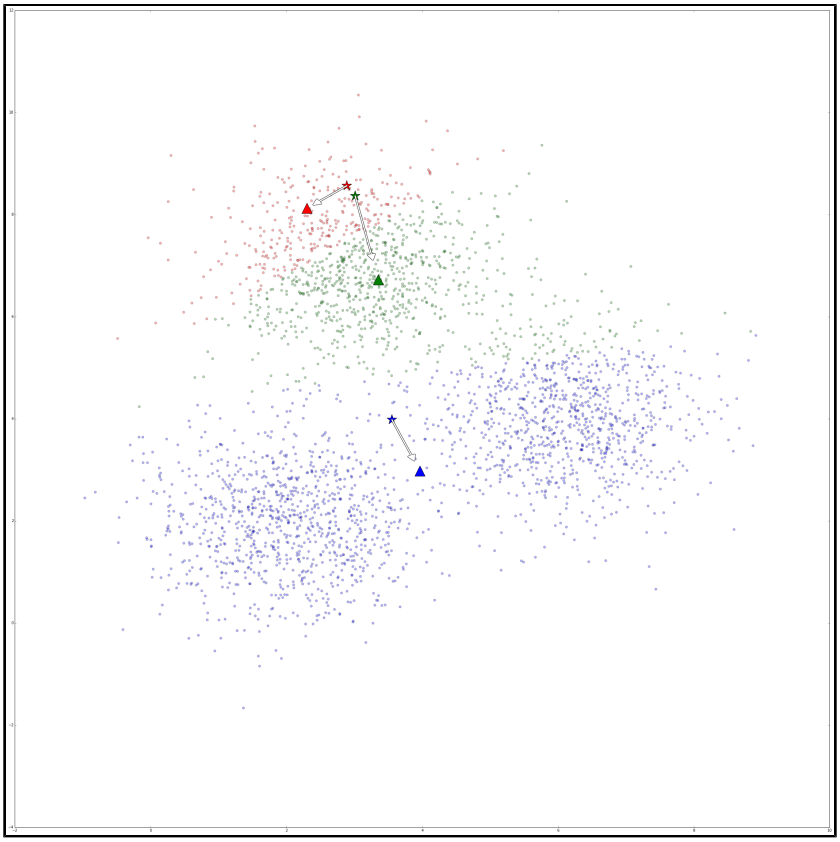
- In the **assignment** step, we set a label to every sample in the dataset linking it to the nearest centroid. For each sample nearest to centroid 1, we assign the label 1. For each sample nearest to centroid 2, we assign a label 2 and so on for each of the k centroids. These labels form the clusters, so we say that each data point with the label 1 is in cluster 1 (at this time only, as assignments can change as the algorithm runs).
- In the **updating** step, we take each of the clusters and compute the centroid, which is the mean of all of the samples in that cluster.

The algorithm then iterates between the assignment step and the updating step; each time the updating step occurs, each of the centroids moves a small amount. This causes the assignments to change slightly, causing the centroids to move a small amount in the next iteration. This repeats until some stopping criterion is reached.



It is common to stop after a certain number of iterations, or when the total movement of the centroids is very low. The algorithm can also complete in some scenarios, which means that the clusters are stable—the assignments do not change and neither do the centroids.

In the following figure, k-means was performed over a dataset created randomly, but with three clusters in the data. The stars represent the starting location of the centroids, which were chosen randomly by picking a random sample from the dataset. Over 5 iterations of the k-means algorithm, the centroids move to the locations represented by the triangles.



The k-means algorithm is fascinating for its mathematical properties and historical significance. It is an algorithm that (roughly) only has a single parameter, and is quite effective and frequently used, even more than 50 years after its discovery.

There is a k-means algorithm in scikit-learn, which we import from the `cluster` module in scikit-learn:

```
from sklearn.cluster import KMeans
```

We also import the `CountVectorizer` class's close cousin, `TfidfVectorizer`. This vectorizer applies a weighting to each term's counts, depending on how many documents it appears in, using the equation: tf / log(df), where tf is a term's frequency (how many times it appears in the current document) and df is the term's document frequency (how many documents in our corpus it appears in). Terms that appear in many documents are weighted lower (by dividing the value by the log of the number of documents it appears in). For many text mining applications, using this type of weighting scheme can improve performance quite reliably. The code is as follows:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

We then set up our pipeline for our analysis. This has two steps. The first is to apply our vectorizer, and the second is to apply our k-means algorithm. The code is as follows:

```
from sklearn.pipeline import Pipeline
n_clusters = 10
pipeline = Pipeline([('feature_extraction', TfidfVectorizer(max_df=0.4)),
                                     ('clusterer', KMeans(n_clusters=n_clusters)) ])
```

The `max_df` parameter is set to a low value of 0.4, which says ignore any word that occurs in more than 40 percent of documents. This parameter is invaluable for removing function words that give little topic-based meaning on their own.

Removing any word that occurs in more than 40 percent of documents will remove function words, making this type of preprocessing quite useless for the work we saw in Chapter 9, Authorship Attribution.

```
documents = [open(os.path.join(text_output_folder, filename)).read()
             for filename in os.listdir(text_output_folder)]
```

We then fit and predict this pipeline. We have followed this process a number of times in this book so far for classification tasks, but there is a difference here—we do not give the target classes for our dataset to the fit function. This is what makes this an unsupervised learning task! The code is as follows:

```
pipeline.fit(documents)
labels = pipeline.predict(documents)
```

The labels variable now contains the cluster numbers for each sample. Samples with the same label are said to belong to the same cluster. It should be noted that the cluster labels themselves are meaningless: clusters 1 and 2 are no more similar than clusters 1 and 3.

We can see how many samples were placed in each cluster using the `Counter` class:

```
from collections import Counter
c = Counter(labels)
for cluster_number in range(n_clusters):
    print("Cluster {} contains {} samples".format(cluster_number, c[cluster_number]))
```

```
Cluster 0 contains 1 samples
Cluster 1 contains 2 samples
Cluster 2 contains 439 samples
Cluster 3 contains 1 samples
Cluster 4 contains 2 samples
Cluster 5 contains 3 samples
Cluster 6 contains 27 samples
Cluster 7 contains 2 samples
Cluster 8 contains 12 samples
Cluster 9 contains 1 samples
```

Many of the results (keeping in mind that your dataset will be quite different to mine) consist of a large cluster with the majority of instances, several medium clusters, and some clusters with only one or two instances. This imbalance is quite normal in many clustering applications.

Evaluating the results

Clustering is mainly an exploratory analysis, and therefore it is difficult to evaluate a clustering algorithm's results effectively. A straightforward way is to evaluate the algorithm based on the criteria the algorithm tries to learn from.

If you have a test set, you can evaluate clustering against it. For more details, visit `http://nlp.stanford.edu/IR-book/html/htmledition/evaluation-of-clustering-1.html`

In the case of the k-means algorithm, the criterion that it uses when developing the centroids is to minimize the distance from each sample to its nearest centroid. This is called the inertia of the algorithm and can be retrieved from any KMeans instance that has had fit called on it:

```
pipeline.named_steps['clusterer'].inertia_
```

The result on my dataset was 343.94. Unfortunately, this value is quite meaningless by itself, but we can use it to determine how many clusters we should use. In the preceding example, we set `n_clusters` to 10, but is this the best value? The following code runs the k-means algorithm 10 times with each value of `n_clusters` from 2 to 20, taking some time to complete the large number of runs.

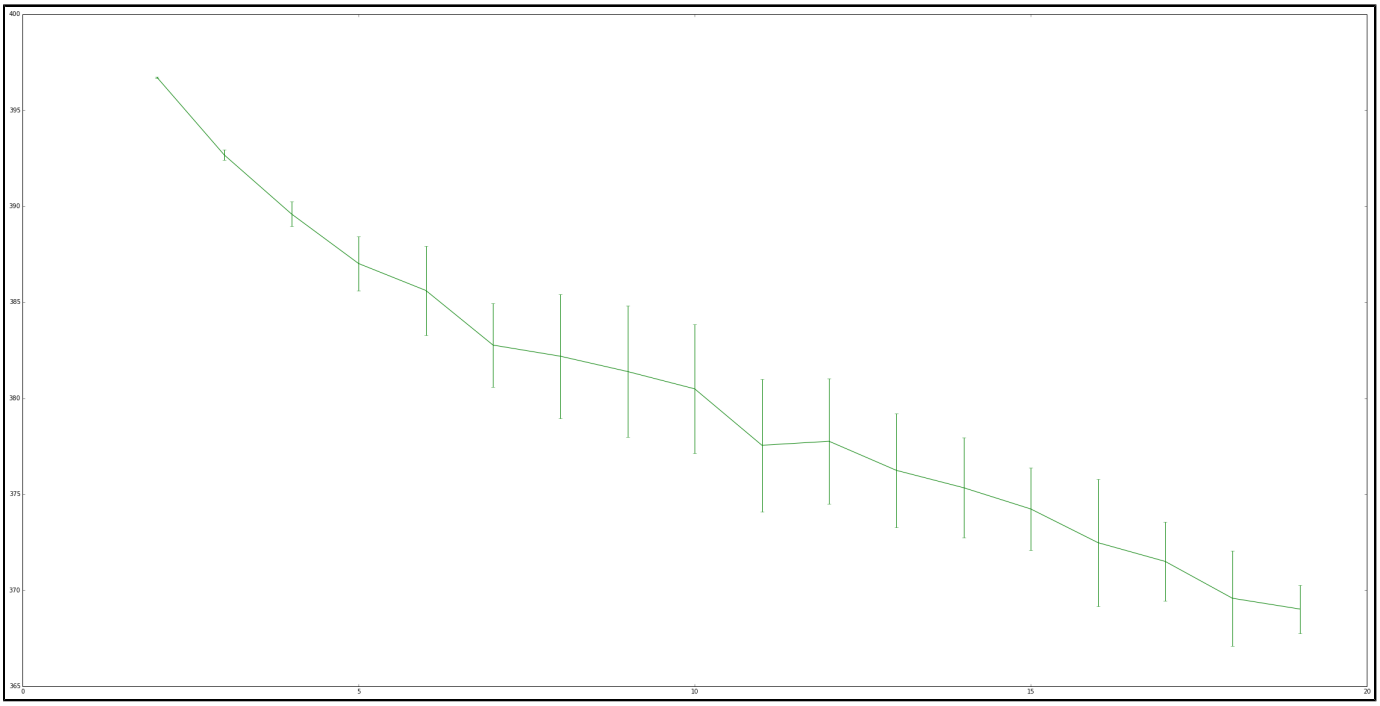
For each run, it records the inertia of the result.

You may notice the following code that we don't use a Pipeline; instead, we split out the steps. We only create the X matrix from our text documents once per value of `n_clusters` to (drastically) improve the speed of this code.

```
inertia_scores = []
n_cluster_values = list(range(2, 20))
for n_clusters in n_cluster_values:
    cur_inertia_scores = []
    X = TfidfVectorizer(max_df=0.4).fit_transform(documents)
 for i in range(10):
        km = KMeans(n_clusters=n_clusters).fit(X)
        cur_inertia_scores.append(km.inertia_)
    inertia_scores.append(cur_inertia_scores)
```

The`inertia_scores` variable now contains a list of inertia scores for each n_clusters value between 2 and 20. We can plot this to get a sense of how this value interacts with `n_clusters`:

```
%matplotlib inline
from matplotlib import pyplot as plt
inertia_means = np.mean(inertia_scores, axis=1)
inertia_stderr = np.std(inertia_scores, axis=1)
fig = plt.figure(figsize=(40,20))
plt.errorbar(n_cluster_values, inertia_means, inertia_stderr, color='green')
plt.show()
```



Overall, the value of the inertia should decrease with reducing improvement as the number of clusters improves, which we can broadly see from these results. The increase between values of 6 to 7 is due only to the randomness in selecting the centroids, which directly affect how good the final results are. Despite this, there is a general trend (for my data; your results may vary) that about 6 clusters was the last time a major improvement in the inertia occurred.

After this point, only slight improvements are made to the inertia, although it is hard to be specific about vague criteria such as this. Looking for this type of pattern is called the elbow rule, in that we are looking for an elbow-esque bend in the graph. Some datasets have more pronounced elbows, but this feature isn't guaranteed to even appear (some graphs may be smooth!).

Based on this analysis, we set `n_clusters` to be 6 and then rerun the algorithm:

```
n_clusters = 6
pipeline = Pipeline([('feature_extraction', TfidfVectorizer(max_df=0.4)),
                     ('clusterer', KMeans(n_clusters=n_clusters)) ])
pipeline.fit(documents)
labels = pipeline.predict(documents)
```

Extracting topic information from clusters

Now we set our sights on the clusters in an attempt to discover the topics in each.

We first extract the term list from our feature extraction step:

```
terms = pipeline.named_steps['feature_extraction'].get_feature_names()
```

We also set up another counter for counting the size of each of our classes:

```
c = Counter(labels)
```

Iterating over each cluster, we print the size of the cluster as before.

It is important to keep in mind the sizes of the clusters when evaluating the results—some of the clusters will only have one sample, and are therefore not indicative of a general trend.

Next (and still in the loop), we iterate over the most important terms for this cluster. To do this, we take the five largest values from the centroid, which we get by finding the features that have the highest values in the centroid itself.

```
for cluster_number in range(n_clusters):
    print("Cluster {} contains {} samples".format(cluster_number, c[cluster_number]))
    print(" Most important terms")
    centroid = pipeline.named_steps['clusterer'].cluster_centers_[cluster_number]
    most_important = centroid.argsort()
    for i in range(5):
        term_index = most_important[-(i+1)]
        print(" {0}) {1} (score: {2:.4f})".format(i+1, terms[term_index], centroid[term_index])
```

The results can be quite indicative of current trends. In my results (obtained January 2017), the clusters correspond to health matters, Middle East tensions, Korean tensions, and Russian affairs. These were the main topics frequenting news around this time—although this has hardly changed for a number of years!

You might notice some words that don't provide much value come out on top, such as *you, her* and *mr.* These function words are great for authorship analysis - as we saw in `Chapter 9`, *Authorship Attribution*, but are not generally very good for topic analysis. Passing the list of function words into the `stop_words` parameter of the **TfidfVectorizer** in our pipeline above will ignore those words. Here is the updated code for building such a pipeline:

```
```

```
function_words = [... list from Chapter 9 ...]
pipeline = Pipeline([('feature_extraction', TfidfVectorizer(max_df=0.4, stop_words=function_wor
                     ('clusterer', KMeans(n_clusters=n_clusters)) ])
```

Using clustering algorithms as transformers

As a side note, one interesting property about the k-means algorithm (and any clustering algorithm) is that you can use it for feature reduction. There are many methods to reduce the number of features (or create new features to embed the dataset on), such as **Principle Component Analysis**, **Latent Semantic Indexing**, and many others. One issue with many of these algorithms is that they often need lots of computing power.

In the preceding example, the terms list had more than 14,000 entries in it—it is quite a large dataset. Our k-means algorithm transformed these into just six clusters. We can then create a dataset with a much lower number of features by taking the distance to each centroid as a feature.

To do this, we call the transform function on a KMeans instance. Our pipeline is fit for this purpose, as it has a k-means instance at the end:

```
X = pipeline.transform(documents)
```

This calls the transform method on the final step of the pipeline, which is an instance of k-means. This results in a matrix that has six features and the number of samples is the same as the length of documents.

You can then perform your own second-level clustering on the result, or use it for classification if you have the target values. A possible workflow for this would be to perform some feature selection using the supervised data, use clustering to reduce the number of features to a more manageable number, and then use the results in a classification algorithm such as SVMs.

Clustering ensembles

In Chapter 3, *Predicting Sports Winners with Decision Trees*, we looked at a classification ensemble using the random forest algorithm, which is an ensemble of many low-quality, tree-based classifiers. Ensembling can also be performed using clustering algorithms. One of the key reasons for doing this is to smooth the results from many runs of an algorithm. As we saw before, the results from running k-means are varied, depending on the selection of the initial centroids. Variation can be reduced by running the algorithm many times and then combining the results.

Ensembling also reduces the effects of choosing parameters on the final result. Most clustering algorithms are quite sensitive to the parameter values chosen for the algorithm. Choosing slightly different parameters results in different clusters.

Evidence accumulation

As a basic ensemble, we can first cluster the data many times and record the labels from each run. We then record how many times each pair of samples was clustered together in a new matrix. This is the essence of the **Evidence Accumulation Clustering** (**EAC**) algorithm.

EAC has two major steps.

1. The first step is to cluster the data many times using a lower-level clustering algorithm, such as k-means and record the frequency that samples were in the same cluster, in each iteration. This is stored in a **co-association matrix**.
2. The second step is to perform a cluster analysis on the resulting co-association matrix, which is performed using another type of clustering algorithm called hierarchical clustering. This has an interesting graph-theory-based property, as it is mathematically the same as finding a tree that links all the nodes together and removing weak links.

We can create a co-association matrix from an array of labels by iterating over each of the labels and recording where two samples have the same label. We use SciPy's `csr_matrix`, which is a type of sparse matrix:

```
from scipy.sparse import csr_matrix
```

Our function definition takes a set of labels and then record the rows and columns of each match. We do these in a list. Sparse matrices are commonly just sets of lists recording the positions of nonzero values, and `csr_matrix` is an example of this type of sparse matrix. For each pair of samples with the same label, we record the position of both samples in our list:

```
import numpy as np
def create_coassociation_matrix(labels):
    rows = []
    cols = []
    unique_labels = set(labels)
    for label in unique_labels:
        indices = np.where(labels == label)[0]
        for index1 in indices:
            for index2 in indices:
                rows.append(index1)
                cols.append(index2)
    data = np.ones((len(rows),))
    return csr_matrix((data, (rows, cols)), dtype='float')
```

To get the co-association matrix from the labels, we simply call this function:

```
C = create_coassociation_matrix(labels)
```

We then remove any node with a weight less than a predefined threshold. To do this, we iterate over the edges in the MST matrix, removing any that are less than a specific value. We can't test this out with just a single iteration in a co-association matrix (the values will be either 1 or 0, so there isn't much to work with). So, we will create extra labels first, create the co-association matrix, and then add the two matrices together. The code is as follows:

```
pipeline.fit(documents)
labels2 = pipeline.predict(documents)
C2 = create_coassociation_matrix(labels2)
C_sum = (C + C2) / 2
```

We then compute the MST and remove any edge that didn't occur in both of these labels:

```
mst = minimum_spanning_tree(-C_sum)
mst.data[mst.data > -1] = 0
```

The threshold we wanted to cut off was any edge not in both clusterings—that is, with a value of 1. However, as we negated the co-association matrix, we had to negate the threshold value too.

Lastly, we find all of the connected components, which is simply a way to find all of the samples that are still connected by edges after we removed the edges with low weights. The first returned value is the number of connected components (that is, the number of clusters) and the second is the labels for each sample. The code is as follows:

```
from scipy.sparse.csgraph import connected_components
number_of_clusters, labels = connected_components(mst)
```

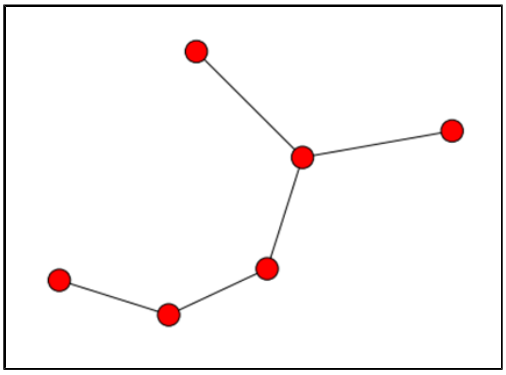
In my dataset, I obtained eight clusters, with the clusters being approximately the same as before. This is hardly a surprise, given we only used two iterations of k-means; using more iterations of k-means (as we do in the next section) will result in more variance.

From here, we can add multiple instances of these matrices together. This allows us to combine the results from multiple runs of k-means. Printing out `C` (just enter C into a new cell of your Jupyter Notebook and run it) will tell you how many cells have nonzero values in them. In my case, about half of the cells had values in them, as my clustering result had a large cluster (the more even the clusters, the lower the number of nonzero values).

The next step involves the hierarchical clustering of the co-association matrix. We will do this by finding minimum spanning trees on this matrix and removing edges with a weight lower than a given threshold.

In graph theory, a spanning tree is a set of edges on a graph that connects all of the nodes together. The **Minimum Spanning Tree** (MST) is simply the spanning tree with the lowest total weight. For our application, the nodes in our graph are samples from our dataset, and the edge weights are the number of times those two samples were clustered together—that is, the value from our co-association matrix.

In the following figure, a MST on a graph of six nodes is shown. Nodes on the graph can be connected to more than once in the MST, as long as all nodes are connected together.



To compute the MST, we use SciPy's `minimum_spanning_tree` function, which is found in the sparse package:

```
from scipy.sparse.csgraph import minimum_spanning_tree
```

The `mst` function can be called directly on the sparse matrix returned by our co-association function:

```
mst = minimum_spanning_tree(C)
```

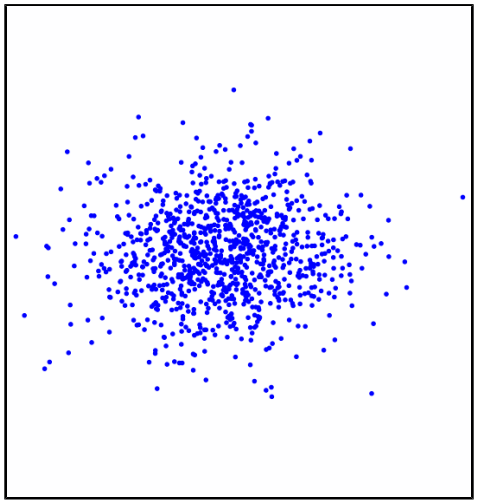
However, in our co-association matrix C, higher values are indicative of samples that are clustered together more often—a similarity value. In contrast, `minimum_spanning_tree` sees the input as a distance, with higher scores penalized. For this reason, we compute the minimum spanning tree on the negation of the co-association matrix instead:

```
mst = minimum_spanning_tree(-C)
```

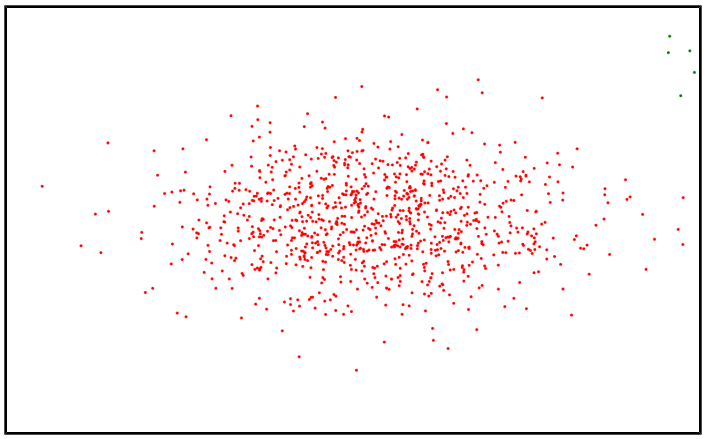
The result from the preceding function is a matrix the same size as the co-association matrix (the number of rows and columns is the same as the number of samples in our dataset), with only the edges in the MST kept and all others removed.

How it works

In the k-means algorithm, each feature is used without any regard to its weight. In essence, all features are assumed to be on the same scale. We saw the problems with not scaling features in Chapter 2*, Classification with scikit-learn Estimators*. The result of this is that k-means is looking for circular clusters, visualized here:

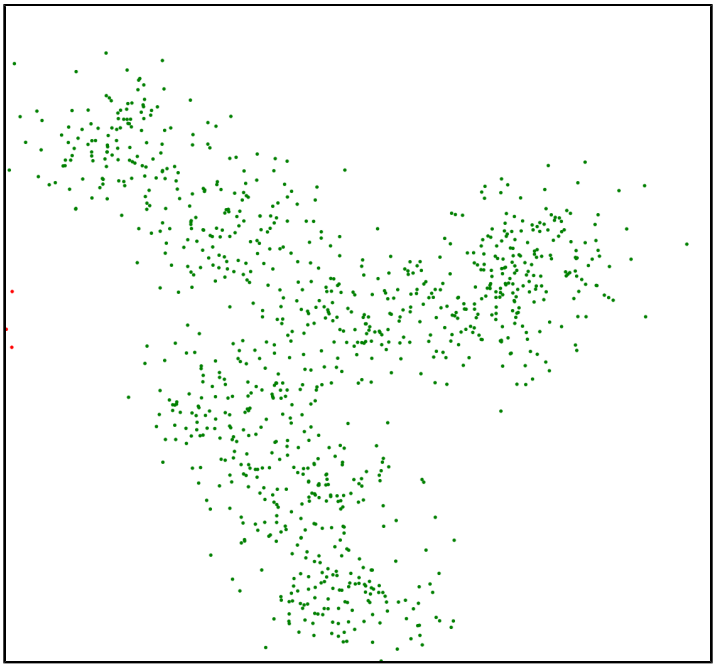


Oval shaped clusters can also be discovered by k-means. The separation usually isn't quite so smooth, but can be made easier with feature scaling. An example of this shaped cluster is as follows:



As we can see in the preceding screenshot, not all clusters have this shape. The blue cluster is circular and is of the type that k-means is very good at picking up. The red cluster is an ellipse. The k-means algorithm can pick up clusters of this shape with some feature scaling.

The bellow third cluster isn't even convex—it is an odd shape that k-means will have trouble discovering, but would still be considered a *cluster*, at least by most humans looking at the picture:



Cluster analysis is a hard task, with most of the difficulty simply in trying to define the problem. Many people intuitively understand what it means, but trying to define it in precise terms (necessary for machine learning) is very difficult. Even people often disagree on the term!

The EAC algorithm works by remapping the features onto a new space, in essence turning each run of the k-means algorithm into a transformer using the same principles we saw the previous section using k-means for feature reduction. In this case, though, we only use the actual label and not the distance to each centroid. This is the data that is recorded in the co-association matrix.

The result is that EAC now only cares about how close things are to each other, not how they are placed in the original feature space. There are still issues around unscaled features. Feature scaling is important and should be done anyway (we did it using tf-idf in this chapter, which results in feature values having the same scale).

We saw a similar type of transformation in `Chapter 9`, *Authorship Attribution*, through the use of kernels in SVMs. These transformations are very powerful and should be kept in mind for complex datasets. The algorithms for remapping data onto a new feature space does not need to be complex though, as you'll see in the EAC algorithm.

Implementation

Putting all this all together, we can now create a clustering algorithm fitting the scikit-learn interface that performs all of the steps in EAC. First, we create the basic structure of the class using scikit-learn's *ClusterMixin*.

Our parameters are the number of k-means clusterings to perform in the first step (to create the co-association matrix), the threshold to cut off at, and the number of clusters to find in each k-means clustering. We set a range of n_clusters in order to get lots of variance in our k-means iterations. Generally, in ensemble terms, variance is a good thing; without it, the solution can be no better than the individual clusterings (that said, high variance is not an indicator that the ensemble will be better).

I'll present the full class first, and then overview each of the functions:

```
from sklearn.base import BaseEstimator, ClusterMixin
class EAC(BaseEstimator, ClusterMixin):
    def __init__(self, n_clusterings=10, cut_threshold=0.5, n_clusters_range=(3, 10)):
        self.n_clusterings = n_clusterings
        self.cut_threshold = cut_threshold
        self.n_clusters_range = n_clusters_range

    def fit(self, X, y=None):
        C = sum((create_coassociation_matrix(self._single_clustering(X))
                 for i in range(self.n_clusterings)))
        mst = minimum_spanning_tree(-C)
        mst.data[mst.data > -self.cut_threshold] = 0
        mst.eliminate_zeros()
        self.n_components, self.labels_ = connected_components(mst)
        return self

    def _single_clustering(self, X):
        n_clusters = np.random.randint(*self.n_clusters_range)
        km = KMeans(n_clusters=n_clusters)
        return km.fit_predict(X)

    def fit_predict(self, X):
        self.fit(X)
        return self.labels_
```

The goal of the `fit` function is to perform the k-means clusters a number of times, combine the co-association matrices and then split it by finding the MST, as we saw earlier with the EAC example. We then perform our low-level clustering using k-means and sum the resulting co-association matrices from each iteration. We do this in a generator to save memory, creating only the co-association matrices when we need them. In each iteration of this generator, we create a new single k-means run with our dataset and then create the co-association matrix for it. We use `sum` to add these together.

As before, we create the MST, remove any edges less than the given threshold (properly negating values as explained earlier), and find the connected components. As with any fit function in scikit-learn, we need to return self in order for the class to work in pipelines effectively.

The `_single_clustering` function is designed to perform a single iteration of k-means on our data, and then return the

predicted labels. To do this, we randomly choose a number of clusters to find using NumPy's `randint` function and our `n_clusters_range` parameter, which sets the range of possible values. We then cluster and predict the dataset using k-means. The return value here will be the labels coming from k-means.

Finally, the `fit_predict` function simply calls fit, and then returns the labels for the documents.

We can now run this on our previous code by setting up a pipeline as before and using EAC where we previously used a KMeans instance as our final stage of the pipeline. The code is as follows:

```
pipeline = Pipeline([('feature_extraction', TfidfVectorizer(max_df=0.4)),
                     ('clusterer', EAC()) ])
```

Online learning

In some cases, we don't have all of the data we need for training before we start our learning. Sometimes, we are waiting for new data to arrive, perhaps the data we have is too large to fit into memory, or we receive extra data after a prediction has been made. In cases like these, online learning is an option for training models over time.

Online learning is the incremental updating of a model as new data arrives. Algorithms that support online learning can be trained on one or a few samples at a time, and updated as new samples arrive. In contrast, algorithms that are not **online** require access to all of the data at once. The standard k-means algorithm is like this, as are most of the algorithms we have seen so far in this book.

Online versions of algorithms have a means to partially update their model with only a few samples. Neural networks are a standard example of an algorithm that works in an online fashion. As a new sample is given to the neural network, the weights in the network are updated according to a learning rate, which is often a very small value such as 0.01. This means that any single instance only makes a small (but hopefully improving) change to the model.

Neural networks can also be trained in batch mode, where a group of samples is given at once and the training is done in one step. Algorithms are generally faster in batch mode but use more memory.

In this same vein, we can slightly update the k-means centroids after a single or small batch of samples. To do this, we apply a learning rate to the centroid movement in the updating step of the k-means algorithm. Assuming that samples are randomly chosen from the population, the centroids should tend to move towards the positions they would have in the standard, offline, and k-means algorithm.

Online learning is related to streaming-based learning; however, there are some important differences. Online learning is capable of reviewing older samples after they have been used in the model, while a streaming-based machine learning algorithm typically only gets one pass—that is, one opportunity to look at each sample.

Implementation

The scikit-learn package contains the **MiniBatchKMeans** algorithm, which allows online learning. This class implements a partial_fit function, which takes a set of samples and updates the model. In contrast, calling fit() will remove any previous training and refit the model only on the new data.

MiniBatchKMeans follows the same clustering format as other algorithms in scikit-learn, so creating and using it is much the same as other algorithms.

The algorithm works by taking a streaming average of all points that it has seen. To compute this, we only need to keep track of two values, which are the current sum of all seen points, and the number of points seen. We can then use this information, combined with a new set of points, to compute the new averages in the updating step.

Therefore, we can create a matrix X by extracting features from our dataset using `TfIDFVectorizer`, and then sample from this to incrementally update our model. The code is as follows:

```
vec = TfidfVectorizer(max_df=0.4)
X = vec.fit_transform(documents)
```

We then import MiniBatchKMeans and create an instance of it:

```
from sklearn.cluster import MiniBatchKMeans
mbkm = MiniBatchKMeans(random_state=14, n_clusters=3)
```

Next, we will randomly sample from our X matrix to simulate data coming in from an external source. Each time we get some data in, we update the model:

```
batch_size = 10
for iteration in range(int(X.shape[0] / batch_size)):
    start = batch_size * iteration
    end = batch_size * (iteration + 1)
    mbkm.partial_fit(X[start:end])
```

We can then get the labels for the original dataset by asking the instance to predict:

```
labels = mbkm.predict(X)
```

At this stage, though, we can't do this in a pipeline as `TfidfVectorizer` is not an online algorithm. To get over this, we use a `HashingVectorizer`. The `HashingVectorizer` class is a clever use of hashing algorithms to drastically reduce the memory of computing the bag-of-words model. Instead of recording the feature names, such as words found in documents, we record only hashes of those names. This allows us to know our features before we even look at the

dataset, as it is the set of all possible hashes. This is a very large number, usually of the order of 2^{18}. Using sparse matrices, we can quite easily store and compute even a matrix of this size, as a very large proportion of the matrix will have the value 0.

Currently, the `Pipeline` class doesn't allow for its use in online learning. There are some nuances in different applications that mean there isn't an obvious one-size-fits-all approach that could be implemented. Instead, we can create our own subclass of Pipeline, which allows us to use it for online learning. We first derive our class from Pipeline, as we only need to implement a single function:

```
class PartialFitPipeline(Pipeline):
    def partial_fit(self, X, y=None):
        Xt = X
        for name, transform in self.steps[:-1]:
            Xt = transform.transform(Xt)
        return self.steps[-1][1].partial_fit(Xt, y=y)
```

The only function we need to implement is the `partial_fit` function, which is performed by first doing all transformation steps, and then calling partial fit on the final step (which should be the classifier or clustering algorithm). All other functions are the same as in the normal Pipeline, class, so we refer (through class inheritance) to those.

We can now create a pipeline to use our `MiniBatchKMeans` in online learning, alongside our `HashingVectorizer`. Other than using our new classes `PartialFitPipeline` and `HashingVectorizer`, this is the same process as used in the rest of this chapter, except we only fit on a few documents at a time. The code is as follows:

```
from sklearn.feature_extraction.text import HashingVectorizer

pipeline = PartialFitPipeline([('feature_extraction', HashingVectorizer()),
                               ('clusterer', MiniBatchKMeans(random_state=14, n_clusters=3)) ])

batch_size = 10
for iteration in range(int(len(documents) / batch_size)):
    start = batch_size * iteration end = batch_size * (iteration + 1)
    pipeline.partial_fit(documents[start:end])
labels = pipeline.predict(documents)
```

There are some downsides to this approach. For one, we can't easily find out which words are most important for each cluster. We can get around this by fitting another `CountVectorizer` and taking the hash of each word. We then look up values by hash rather than word. This is a bit cumbersome and defeats the memory gains from using HashingVectorizer. Further, we can't use the `max_df` parameter that we used earlier, as it requires us to know what the features mean and to count them over time.

We also can't use tf-idf weighting when performing training online. It would be possible to approximate this and apply such weighting, but again this is a cumbersome approach. `HashingVectorizer` *is still a very useful algorithm and a great use of hashing algorithms.*

Summary

In this chapter, we looked at clustering, which is an unsupervised learning approach. We use unsupervised learning to explore data, rather than for classification and prediction purposes. In the experiment here, we didn't have topics for the news items we found on reddit, so we were unable to perform classification. We used k-means clustering to group together these news stories to find common topics and trends in the data.

In pulling data from reddit, we had to extract data from arbitrary websites. This was performed by looking for large text segments, rather than a full-blown machine learning approach. There are some interesting approaches to machine learning for this task that may improve upon these results. In the Appendix of this book, I've listed, for each chapter, avenues for going beyond the scope of the chapter and improving upon the results. This includes references to other sources of information and more difficult applications of the approaches in each chapter.

We also looked at a straightforward ensemble algorithm, EAC. An ensemble is often a good way to deal with variance in the results, especially if you don't know how to choose good parameters (which is always difficult with clustering).

Finally, we introduced online learning. This is a gateway to larger learning exercises, including big data, which will be discussed in the final two chapters of this book. These final experiments are quite large and require management of data as well as learning a model from them.

As an extension on the work in this chapter, try implementing EAC to be an online learning algorithm. This is not a trivial task and will involve some thought on what should happen when the algorithm is updated. Another extension is to collect more data from more data sources (such as other subreddits or directly from news websites or blogs) and look for general trends.

In the next chapter, we'll step away from unsupervised learning and go back to classification. We will look at deep learning, which is a classification method built on complex neural networks.